

# Menguji Keterbatasan YOLO pada Objek Kecil Berkelompok: Pengaruh Resolusi Input 640 vs 1280 pada VisDrone2019-DET

Royan Aditya

Program Studi Magister Kecerdasan Artifisial, Departemen Ilmu Komputer dan Elektronika,  
Fakultas Matematika dan Ilmu Pengetahuan Alam  
Universitas Gadjah Mada, Yogyakarta, Indonesia

Notebook Google Colab (akses "Anyone with the link can view"):

<https://colab.research.google.com/drive/1OSrupChmztcs8s-jjMuyXK7diRoSMSBo?usp=sharing>

**Abstrak-** Deteksi objek kecil pada citra udara merupakan salah satu tantangan dalam deteksi objek karena objek memiliki representasi visual yang terbatas dan sering muncul dalam jumlah banyak serta berkelompok. Permasalahan tersebut terlihat pada VisDrone2019-DET, yang pada test-dev terdiri atas 1.610 citra dengan 75.102 objek, dengan 67,7% di antaranya merupakan objek kecil. Penelitian terdahulu oleh Redmon dkk. [1] menunjukkan bahwa YOLO mengalami kesulitan dalam mendeteksi objek kecil yang berkelompok akibat keterbatasan prediksi berbasis grid. Eksperimen ini menguji apakah permasalahan tersebut masih terlihat pada detektor modern YOLO11n serta menganalisis pengaruh peningkatan resolusi input terhadap deteksi objek kecil. Dua konfigurasi dibandingkan menggunakan seed, hiperparameter, dan jumlah epoch yang identik, yaitu YOLO11n dengan resolusi  $640 \times 640$  sebagai baseline dan  $1280 \times 1280$  sebagai intervensi. Pada resolusi 640, AP@0,5 objek kecil sebesar 0,080, lebih rendah dibandingkan objek sedang sebesar 0,285 dan objek besar sebesar 0,455. Peningkatan resolusi menjadi 1280 meningkatkan AP@0,5 objek kecil menjadi 0,181 atau sebesar 125% secara relatif. Namun, peningkatan tersebut disertai kenaikan waktu pelatihan sebesar  $2,23\times$  dan penurunan FPS dari 52,8 menjadi 40,2. Objek kecil tetap menjadi kategori dengan AP terendah, sementara objek sedang dan besar juga mengalami peningkatan. Hasil tersebut menunjukkan bahwa peningkatan resolusi dapat meningkatkan kemampuan deteksi objek kecil, tetapi belum sepenuhnya mengatasi permasalahan deteksi objek kecil dan berkonsekuensi terhadap biaya komputasi. Eksperimen ini belum dapat memisahkan pengaruh resolusi input dari pengaruh perubahan kepadatan grid.

**Kata kunci-** YOLO11n, objek kecil, VisDrone2019-DET, resolusi input, AP per ukuran, deteksi objek.

## I. PENDAHULUAN

### A. Latar Belakang

Deteksi objek pada citra udara dari drone menghadapi tantangan berupa ukuran objek yang kecil dan kepadatan objek yang tinggi. Kondisi tersebut menyebabkan objek memiliki representasi visual yang terbatas sehingga lebih sulit dibedakan dari latar belakang maupun objek lain di sekitarnya. Permasalahan ini menjadi semakin relevan pada VisDrone2019-DET, yang berisi citra udara dengan objek berukuran beragam dan didominasi oleh objek kecil. Pada test-dev dataset tersebut terdapat 1.610 citra dengan 75.102 objek, dan sekitar 67,7% objek termasuk kategori kecil.

Permasalahan objek kecil telah menjadi salah satu keterbatasan yang dibahas pada detektor YOLO. Redmon dkk. [1] menjelaskan bahwa formulasi YOLO sebagai prediksi berbasis grid dapat mengalami kesulitan ketika mendeteksi

objek kecil yang muncul secara berkelompok. Pada YOLOv1, setiap sel pada grid bertanggung jawab terhadap prediksi objek tertentu sehingga keterbatasan representasi spasial dapat menjadi masalah ketika beberapa objek berada dalam area yang berdekatan. Dengan konfigurasi  $S = 7$  dan  $B = 2$ , YOLOv1 menghasilkan 49 sel grid dengan dua prediksi kotak pada setiap sel.

Meskipun demikian, detektor YOLO telah berkembang secara signifikan sejak YOLOv1. Salah satu detektor modern adalah YOLO11, yang menggunakan arsitektur deteksi yang berbeda dari YOLOv1. Perkembangan tersebut menimbulkan pertanyaan apakah kesulitan mendeteksi objek kecil yang dilaporkan pada YOLOv1 masih dapat diamati pada detektor modern. Permasalahan tersebut penting untuk diuji karena karakteristik objek kecil dan berkelompok masih banyak ditemukan pada citra udara.

Salah satu pendekatan yang dapat digunakan untuk menghadapi objek kecil adalah meningkatkan resolusi input. Peningkatan resolusi memungkinkan objek berukuran kecil direpresentasikan oleh jumlah piksel yang lebih banyak sehingga informasi visual yang tersedia bagi model juga meningkat. Namun, peningkatan resolusi juga meningkatkan beban komputasi, sehingga perbaikan kemampuan deteksi perlu dilihat bersama dengan perubahan waktu pelatihan dan kecepatan inferensi.

Berdasarkan permasalahan tersebut, penelitian ini menguji kemampuan YOLO11n dalam mendeteksi objek berdasarkan ukurannya pada dataset VisDrone2019-DET. Eksperimen dilakukan dengan membandingkan dua konfigurasi, yaitu resolusi input  $640 \times 640$  sebagai baseline dan  $1280 \times 1280$  sebagai intervensi. Kedua konfigurasi menggunakan seed, hiperparameter, dan jumlah epoch yang sama sehingga perbedaan hasil terutama diamati dari perubahan resolusi input.

Evaluasi tidak hanya menggunakan metrik keseluruhan, tetapi juga membandingkan AP@0,5 untuk objek kecil, sedang, dan besar serta biaya komputasi berupa waktu pelatihan dan FPS. Dengan demikian, eksperimen dapat menunjukkan apakah masalah deteksi objek kecil masih terlihat pada YOLO11n dan sejauh mana peningkatan resolusi dapat memperbaikinya.

### B. Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam eksperimen ini meliputi:

1. Kinerja YOLO11n dalam mendeteksi objek kecil, sedang, dan besar pada dataset VisDrone2019-DET.

2. Pengaruh peningkatan resolusi input dari  $640 \times 640$  menjadi  $1280 \times 1280$  terhadap kemampuan YOLO11n dalam mendeteksi objek kecil.
3. Perubahan biaya komputasi akibat peningkatan resolusi input, yang meliputi waktu pelatihan dan kecepatan inferensi.

### C. Tujuan

Berdasarkan rumusan masalah tersebut, tujuan eksperimen ini adalah:

1. Menganalisis kinerja YOLO11n dalam mendeteksi objek kecil, sedang, dan besar pada dataset VisDrone2019-DET.
2. Menganalisis pengaruh peningkatan resolusi input dari  $640 \times 640$  menjadi  $1280 \times 1280$  terhadap kemampuan deteksi objek kecil.
3. Menganalisis perubahan biaya komputasi akibat peningkatan resolusi input berdasarkan waktu pelatihan dan kecepatan inferensi.

## II. METODE

### A. Ringkasan metode paper

Citra dibagi menjadi grid  $S \times S$ . Sel yang memuat titik pusat objek bertanggung jawab atas objek itu dan memprediksi  $B$  kotak ( $x, y, w, h, confidence$ ) serta  $C$  probabilitas kelas, sehingga keluaran berukuran  $S \times S \times (5B+C)$ . Fungsi loss YOLOv1 (disederhanakan ke tingkat sel) adalah

$$\begin{aligned} \mathcal{L} = & \lambda_{\text{coord}} \sum_i \mathbf{1}_i^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \\ & + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\ & + \sum_i \mathbf{1}_i^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_i \mathbf{1}_i^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_i \mathbf{1}_i^{\text{obj}} \sum_c (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (1)$$

dengan  $\lambda_{\text{coord}} = 5$  dan  $\lambda_{\text{noobj}} = 0,5$ . Akar pada  $w, h$  membuat galat kotak kecil lebih terasa. Karena satu sel hanya memuat satu objek, objek kedua pada sel yang sama tidak dapat dikodekan.

### B. Bagian yang diimplementasikan

#### (1) Aturan satu objek per sel.

Fungsi `encode_yolov1_target` dan `yolov1_loss` (diadaptasi dari notebook tutorial Sesi 3, dengan penghitung objek terbuang) mengimplementasikan penyandian target dan Persamaan (1). Sebagai uji kewajaran, pada satu citra latih (57 objek) loss total prediksi acak adalah 131,589 sedangkan prediksi yang mendekati target 0,194. Fungsi ini dipakai untuk mengukur berapa persen anotasi VisDrone yang terbuang pada grid berukuran  $S$  (Tabel III), yaitu proksi tingkat “berkelompok”.

### (2) Pengujian empiris.

YOLO11n dilatih dan diuji pada dua resolusi. AP per ukuran dihitung dengan prosedur COCO: untuk rentang luas  $s \in \{\text{small, medium, large}\}$ , GT di luar rentang diabaikan, dan deteksi tak cocok yang luasnya di luar rentang juga diabaikan,

$$\text{AP}_s = \frac{1}{|C|} \sum_{c \in C} \frac{1}{101} \sum_{r \in R} \max_{r' \geq r} p_{c,s}(r') \quad (2)$$

dengan  $R = \{0; 0,01; \dots; 1\}$  dan  $p_{c,s}$  kurva presisi–recall kelas  $c$  pada ukuran  $s$ .  $\text{mAP}@0,5:0,95$  adalah rata-rata AP pada IoU 0,50 hingga 0,95 (langkah 0,05). Ukuran mengikuti COCO [4] dalam piksel citra asli: kecil  $< 32^2$ , sedang  $32^2-96^2$ , besar  $> 96^2$ . Selisih antar kondisi dilaporkan sebagai

$$\begin{aligned} \Delta_{\text{abs}} &= \text{AP}_B - \text{AP}_A, \\ \Delta_{\%} &= 100 \frac{\text{AP}_B - \text{AP}_A}{\text{AP}_A}. \end{aligned} \quad (3)$$

Untuk menelusuri sumber kegagalan dipakai recall pada skor  $\geq 0,25$  dan IoU  $\geq 0,5$  dengan pencocokan greedy (skor tertinggi dulu), dalam dua versi: sadar-kelas dan agnostik-kelas. Selisihnya menunjukkan kegagalan karena salah kelas.

### C. Perbedaan dengan paper

Tabel I merangkum perbedaannya. Angka mAP VOC milik paper tidak dapat dibandingkan langsung dengan angka di sini, sehingga yang dibandingkan adalah tren kualitatif: apakah objek kecil tetap paling lemah.

TABEL I  
PERBEDAAN PAPER DAN EKSPERIMEN INI

| Aspek      | Paper (YOLOv1)                      | Eksperimen ini (YOLO11n)                    | Alasan                                                        |
|------------|-------------------------------------|---------------------------------------------|---------------------------------------------------------------|
| Arsitektur | Grid $7 \times 7$ , 2 kotak per sel | Multi-skala P3–P5, tanpa anchor             | Menguji apakah keterbatasan masih berlaku pada YOLO modern    |
| Resolusi   | $448 \times 448$                    | 640 vs 1280                                 | Intervensi yang diminta penugasan                             |
| Data       | PASCAL VOC                          | VisDrone2019-DET                            | Objek kecil berkelompok; disarankan lembar tugas              |
| Metrik     | $\text{mAP}@0,5$ (VOC)              | $\text{mAP}@0,5$ dan $@0,5:0,95$ per ukuran | Metrik wajib Opsi C; dapat memisahkan ukuran objek            |
| Bobot awal | ImageNet                            | COCO ( <code>yolo11n.pt</code> )            | Data latih hanya sub-sampel, sehingga butuh inisialisasi kuat |
| Pelatihan  | Seluruh data latih VOC              | 2.000 dari 6.471 citra, 30 epoch            | Batas waktu dan memori Colab (GPU T4)                         |

## III. SETUP EKSPERIMEN

### A. Dataset dan pembagian data

VisDrone2019-DET [2] memiliki 10 kelas: pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, dan motor. Berkas ZIP train, val, dan test-dev diunduh otomatis dari GitHub Releases Ultralytics, lalu anotasi dikonversi ke format YOLO. Baris dengan skor 0 (*ignored regions*) dan kategori *others* dibuang (3.243, 1.410, dan 2.445 baris pada train, val, dan test). Citra latih adalah sub-sampel acak (seed 42) sebanyak 2.000 dari 6.471; val (548 citra) dipakai memilih checkpoint; test-dev (1.610 citra) tidak dipakai saat pelatihan. Ukuran citra bervariasi ( $960 \times 540$  hingga  $2000 \times 1500$ ); pada test, 933 citra berukuran  $1400 \times 788$  dan 274 berukuran  $1400 \times 1050$ .

TABEL II

RINGKASAN DATA (§2 NOTEBOOK); OBJ/CITRA = RATA-RATA / MAKSIMUM

| Split        | Citra | Objek   | Kecil % | Sedang % | Besar % | Obj/citra  |
|--------------|-------|---------|---------|----------|---------|------------|
| Train (sub.) | 2.000 | 105.879 | 60,24   | 34,03    | 5,74    | 52,9 / 370 |
| Val          | 548   | 38.759  | 68,59   | 28,65    | 2,76    | 70,7 / 317 |
| Test-dev     | 1.610 | 75.102  | 67,69   | 29,10    | 3,21    | 46,6 / 461 |

TABEL III

GT TERBUANG (%) PADA ATURAN SATU OBJEK PER SEL (§3.2)

| S   | Train | Val   | Test  |
|-----|-------|-------|-------|
| 7   | 70,25 | 74,05 | 66,49 |
| 20  | 38,84 | 40,25 | 35,35 |
| 40  | 20,43 | 20,38 | 18,75 |
| 80  | 8,14  | 7,76  | 8,11  |
| 160 | 2,42  | 1,96  | 2,44  |

### B. Model, baseline, dan parameter

Baseline (kondisi A) adalah YOLO11n berbobot awal COCO pada 640×640; intervensi (kondisi B) identik tetapi pada 1280×1280, untuk latih maupun uji. Parameter yang sama pada keduanya: 30 epoch, *patience* 1000 (praktis tanpa early stopping, agar jumlah epoch sama), batch 2, seed 42, *deterministic=True*, serta optimizer dan augmentasi default Ultralytics. Evaluasi test memakai skor minimum 0,001, IoU NMS 0,7, dan maksimum 500 deteksi per citra. Karena hanya *imgsz* yang berbeda, baseline ini adil untuk RQ2; baseline YOLOv1 asli tidak dilatih.

### C. Metrik dan komputasi

Metrik utama: mAP@0,5 dan mAP@0,5:0,95 (Ultralytics), AP dan AR@500 per ukuran (pycocotools, skor  $\geq 0,01$ ), recall dan TP/FP/FN pada skor  $\geq 0,25$ , serta biaya: menit latih, ms per citra (batch 1), dan FPS. mAP@0,5 juga dihitung dengan fungsi AP manual (interpolasi semua titik ala VOC [5]) sebagai pembandingan silang. Eksperimen berjalan di Google Colab dengan GPU Tesla T4 (15,6 GB), Python 3.13.15, PyTorch 2.11.0+cu128, dan Ultralytics 8.4.166.

## IV. HASIL

### A. Seberapa berkelompok data ini

Tabel III menunjukkan persentase GT yang terbuang bila anotasi dikodekan dengan aturan satu objek per sel. Pada  $S = 7$  (YOLOv1), 66,5% anotasi test tidak dapat dikodekan; rata-rata ada 46,6 objek per citra test dan maksimum 461, jauh di atas batas 49 objek dan 98 kotak.  $S = 80$  dan  $S = 160$  kira-kira setara peta fitur stride 8 (P3) pada input 640 dan 1280.

### B. Hasil keseluruhan, biaya, dan pembandingan silang

Tabel IV memperlihatkan bahwa 1280 meningkatkan seluruh metrik deteksi, dengan biaya latih 2,23× dan FPS turun 23,9%. Jumlah parameter sama (2,59 juta).

TABEL IV

HASIL TEST-DEV DAN BIAYA, ULTRALYTICS (§4)

| Metrik               | A (640) | B (1280) | Selish  |
|----------------------|---------|----------|---------|
| mAP@0,5              | 0,2001  | 0,3249   | +62,4%  |
| mAP@0,5:0,95         | 0,1079  | 0,1869   | +73,2%  |
| Presisi (P)          | 0,3016  | 0,4259   | +0,1243 |
| Recall (R)           | 0,2510  | 0,3580   | +0,1070 |
| Waktu latih (menit)  | 27,32   | 60,88    | ×2,23   |
| Inferensi (ms/citra) | 15,02   | 18,55    | ×1,24   |
| Total (ms/citra)     | 18,92   | 24,88    | ×1,31   |
| FPS                  | 52,84   | 40,19    | -23,9%  |

Angka mAP@0,5 keseluruhan berbeda antar implementasi (Tabel V): Ultralytics lebih tinggi sekitar 0,026–0,027 daripada pycocotools dan AP manual, yang dua-duanya saling berdekatan. Arah dan urutan kondisi sama pada ketiganya, tetapi kenaikan relatifnya berkisar +62% hingga +72%. Perbandingan A–B selalu memakai pipeline yang sama.

TABEL V

PEMBANDING SILANG MAP@0,5 TEST-DEV (§5.2)

| Sumber              | A (640) | B (1280) | $\Delta\%$ |
|---------------------|---------|----------|------------|
| Ultralytics         | 0,2001  | 0,3249   | +62,4      |
| pycocotools (semua) | 0,1744  | 0,2974   | +70,5      |
| AP manual (VOC)     | 0,1731  | 0,2984   | +72,4      |

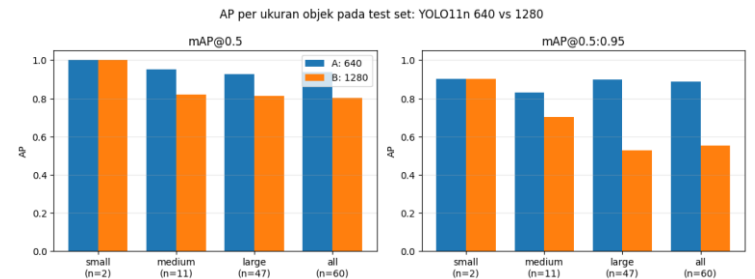
### C. Hasil per ukuran objek

Tabel VI dan Gambar 1 menyajikan AP per ukuran. Pada kondisi A, AP@0,5 turun tajam dari objek besar (0,455) ke sedang (0,285) dan kecil (0,080). Resolusi 1280 menaikkan AP objek kecil sebesar +0,101 (+125,5%), tetapi kenaikan absolut pada objek sedang (+0,161) lebih besar.

TABEL VI

AP DAN AR PER UKURAN OBJEK PADA TEST-DEV, PYCOCOTOOLS (§5)

| Ukuran | $n_{GT}$ | (a) AP@0,5 |        |                |            | (b) AP@0,5:0,95 |        |                |            | AR@500 |        |
|--------|----------|------------|--------|----------------|------------|-----------------|--------|----------------|------------|--------|--------|
|        |          | A          | B      | $\Delta_{abs}$ | $\Delta\%$ | A               | B      | $\Delta_{abs}$ | $\Delta\%$ | A      | B      |
| Kecil  | 50.837   | 0,0802     | 0,1808 | +0,1006        | +125,5     | 0,0328          | 0,0837 | +0,0508        | +154,8     | 0,0723 | 0,1826 |
| Sedang | 21.856   | 0,2847     | 0,4457 | +0,1610        | +56,5      | 0,1572          | 0,2698 | +0,1126        | +71,6      | 0,2824 | 0,4435 |
| Besar  | 2.409    | 0,4546     | 0,5517 | +0,0971        | +21,4      | 0,2921          | 0,3965 | +0,1043        | +35,7      | 0,4317 | 0,5747 |
| Semua  | 75.102   | 0,1744     | 0,2974 | +0,1230        | +70,5      | 0,0974          | 0,1743 | +0,0770        | +79,0      | 0,1729 | 0,2966 |



Gambar 1. AP per ukuran objek pada test-dev: YOLO11n 640 (A) vs 1280 (B). Angka  $n$  adalah jumlah GT. Dihasilkan oleh §5 notebook.

### D. Recall dan objek terlewat

Tabel VII merinci recall pada skor  $\geq 0,25$ . Dari 50.837 objek kecil, kondisi A menemukan hanya 17,9% (sadar-kelas), dan

1280 menaikannya menjadi 33,8%. Pada skor yang sama, jumlah objek kecil yang terlewat turun dari 41.752 menjadi 33.671 (-19,4%), sedangkan objek sedang turun -30,1% dan besar -25,0%. Jumlah FP naik dari 17.317 ke 21.047 (+21,5%) sementara TP naik 45,8% (23.835 ke 34.748); presisi total pada skor 0,25 naik dari 0,579 ke 0,623.

TABEL VII  
RECALL (SKOR  $\geq 0,25$ , IOU  $\geq 0,5$ ) DAN OBJEK TERLEWAT (FN) PER UKURAN (§6)

| Ukuran | $n_{GT}$ | Recall A | Recall B | Agnostik A | Agnostik B | FN A   | FN B   |
|--------|----------|----------|----------|------------|------------|--------|--------|
| Kecil  | 50.837   | 0,1787   | 0,3377   | 0,2156     | 0,3923     | 41.752 | 33.671 |
| Sedang | 21.856   | 0,5928   | 0,7154   | 0,7004     | 0,8112     | 8.899  | 6.221  |
| Besar  | 2.409    | 0,7443   | 0,8082   | 0,8294     | 0,8809     | 616    | 462    |
| Semua  | 75.102   | 0,3174   | 0,4627   | 0,3764     | 0,5299     | 51.267 | 40.354 |

## V. ANALISIS

### A. Apakah klaim terbukti? (RQ1)

**Pola yang dilaporkan paper tetap terlihat.** Pada kondisi A, AP@0,5 kecil:sedang:besar adalah 0,080:0,285:0,455, AP@0,5:0,95 adalah 0,033:0,157:0,292, dan recall pada skor 0,25 adalah 0,179:0,593:0,744. Objek kecil mencakup 67,7% GT tetapi 81,4% dari seluruh FN kondisi A (41.752 dari 51.267). Jadi YOLO11n tetap paling lemah pada objek kecil.

**Namun mekanisme grid-nya tidak cukup menjelaskan.** Pada  $S = 80$  (P3 pada 640), hanya 8,1% GT test yang berbagi sel dengan GT lain (Tabel III), sedangkan 82,1% objek kecil tidak terdeteksi pada skor 0,25. Tabrakan sel gaya YOLOv1 karena itu tidak dapat menjadi penyebab utama pada YOLO11n. Aturan satu objek per sel memang menghapus 66,5% anotasi pada  $S = 7$ , yang mendukung klaim paper untuk arsitektur YOLOv1 menurut data ini, tetapi itu hasil penyandian data dan bukan hasil melatih YOLOv1. Penyebab yang lebih masuk akal untuk YOLO11n adalah (dugaan, belum diuji dengan ablasi) detail piksel yang hilang akibat pengecilan citra, kapasitas model kecil (2,6 juta parameter), dan pelatihan yang belum konvergen: epoch terbaik pada validasi adalah epoch 30 untuk kedua kondisi, yang mengindikasikan pelatihan belum konvergen.

**Ukuran bercampur dengan kelas.** Objek kecil didominasi pedestrian (46.092), car (43.775), dan people (17.918), sehingga AP kecil sebagian mencerminkan kelas-kelas tersebut, bukan ukuran semata.

### B. Apakah resolusi 1280 membantu? (RQ2)

Ya untuk arah, tetapi tidak menutup masalah. AP@0,5 objek kecil naik dari 0,080 ke 0,181 dan AP@0,5:0,95 dari 0,033 ke 0,084. Meski begitu, AP@0,5 kecil pada 1280 (0,181) masih di bawah AP@0,5 sedang pada 640 (0,285), dan rasio kecil terhadap besar baru naik dari 0,18 ke 0,33. Kenaikan absolut terbesar justru pada objek sedang (+0,161), jadi resolusi tinggi bukan perbaikan yang khusus untuk objek kecil. Pada  $n_{GT} = 50.837$  selisihnya besar untuk ukuran sampel itu, tetapi dengan satu seed per kondisi selang kepercayaan tidak dapat dihitung; arah ( $B > A$ ) cukup kuat, besarnya belum.

Satu penjelasan yang konsisten dengan data (dugaan): sebagian besar citra test lebih lebar dari 1280 piksel, jadi pada 640 sisi terpanjang menyusut sekitar  $2,2\times$  untuk citra 1400 piksel dan objek 32 piksel menjadi sekitar 15 piksel, sedangkan

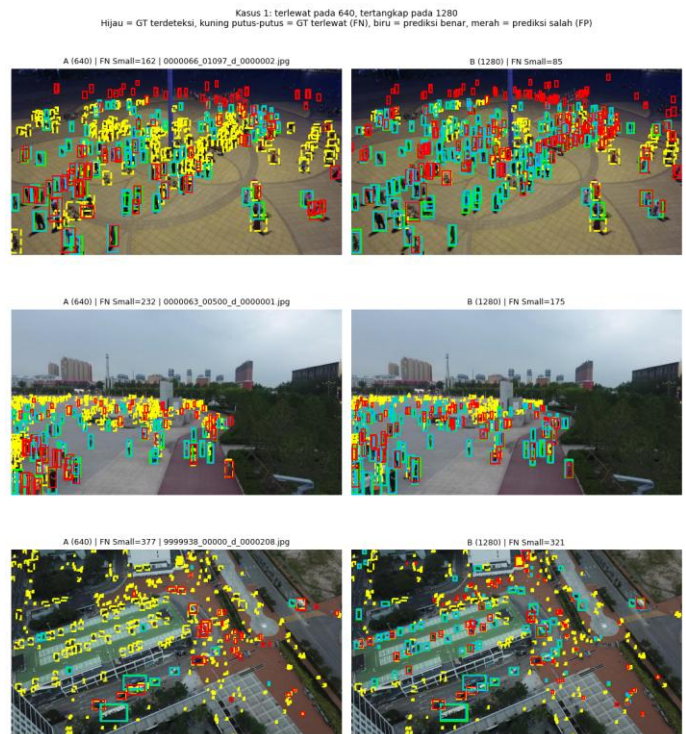
pada 1280 penyusutannya sekitar  $1,1\times$ . Pengamatan ini membalik peringatan notebook tentang *upsampling*: pada VisDrone, resolusi 1280 terutama *mengurangi downsampling*. Di sisi lain, kerapatan grid juga naik bersamaan (Tabel III: 8,1% menjadi 2,4%), sehingga desain ini tidak memisahkan kedua faktor.

**Biaya.** Waktu latih naik  $2,23\times$  (27,3 menjadi 60,9 menit), latensi total  $1,31\times$  (18,9 menjadi 24,9 ms), dan FPS turun dari 52,8 ke 40,2. Jumlah piksel naik  $4\times$  tetapi latensi inferensi hanya  $1,24\times$ ; dugaan kami GPU belum jenuh pada batch 1 dan 640. Pengukuran ms per citra berasal dari evaluasi batch 1 sehingga tidak mewakili throughput batch besar.

### C. Sumber kegagalan dan kasus gagal

Selisih recall sadar-kelas dan agnostik pada objek kecil hanya 3,7 poin (A) dan 5,5 poin (B), sedangkan pada objek sedang 10,8 dan 9,6 poin, dan pada besar 8,5 dan 7,3 poin. Maka kegagalan objek kecil terutama berupa *tidak ada kotak sama sekali* dengan IOU  $\geq 0,5$ : 78,4% (A) dan 60,8% (B) objek kecil tidak tersentuh oleh kotak kelas apa pun, sedangkan salah kelas lebih berperan pada objek sedang dan besar.

Gambar 2 menampilkan tiga pola dari kategori ukuran dengan FN terbanyak (kecil). Kasus diambil dari yang paling ekstrem menurut kriteria, bukan acak.



Gambar 2. Kasus gagal (baris atas: A 640, baris bawah: B 1280; hijau = GT terdeteksi, kuning putus-putus = GT terlewat, biru = prediksi benar, merah = FP). Kolom 1: terlewat pada 640 dan sebagian tertangkap pada 1280 (FN kecil 162  $\rightarrow$  85). Kolom 2: terlewat pada kedua resolusi (338  $\rightarrow$  291). Kolom 3: 1280 lebih buruk (17  $\rightarrow$  20). Dipotong dari keluaran §6 notebook.

**Kasus 1: kerumunan padat (kolom 1).** Pada plaza malam hari, FN kecil turun dari 162 ke 85 (-47,5%); banyak pejalan kaki yang terlewat pada 640 tertangkap pada 1280. Namun pada 1280 muncul lebih banyak kotak merah di area yang sangat rapat, yaitu duplikat atau kotak yang bergeser. Resolusi

memulihkan instans yang hilang, tetapi kerumunan lalu menimbulkan galat lokalisasi.

**Kasus 2: objek kecil rapat pada ketinggian tinggi (kolom 2).** Pada tiga citra berurutan dari satu urutan video (kendaraan parkir berbaris dan pejalan kaki sangat kecil), FN kecil hanya turun 10–15% (442 → 399, 377 → 321, 338 → 291). Sebagian besar GT tetap terlewat pada 1280. Pada citra 0210, kondisi A hanya menyisakan kurang dari 30 prediksi berskor  $\geq 0,25$  untuk ratusan GT, artinya skor kepercayaan objek-objek ini jarang melewati ambang. Ini paling dekat dengan skenario “objek kecil berkelompok” pada paper, dan resolusi saja tidak cukup. Ketiga citra berasal dari satu urutan sehingga bukan bukti yang independen.

**Kasus 3: regresi pada 1280 (kolom 3).** Tiga regresi terbesar (FN kecil 13 → 18, 17 → 20, 8 → 10) hanya menambah 2–5 FN pada citra yang memuat sedikit objek kecil, disertai tambahan FP pada kendaraan atau pejalan kaki yang lebih besar. Regresinya kecil dibanding penurunan 8.081 FN kecil secara keseluruhan, tetapi proporsi citra yang mengalami regresi tidak dikuantifikasi di laporan ini.

#### D. Mengapa berbeda dari paper?

Paper menguji YOLOv1 (grid  $7 \times 7$ , VOC,  $448 \times 448$ ) sehingga keterbatasannya bersifat struktural. YOLO11n memakai tiga skala dan tanpa anchor, sehingga batas 49 objek tidak berlaku, namun pola kelemahan pada objek kecil tetap ada. Karena arsitektur, data, dan metrik berbeda, kesamaan pola ini tidak membuktikan bahwa penyebabnya sama.

## VI. KESIMPULAN DAN KETERBATASAN

**RQ1.** Terbukti dalam arti pola: AP objek kecil paling rendah (0,080 vs 0,285 dan 0,455 pada AP@0,5), meski sebab strukturalnya (satu objek per sel) tidak cukup menjelaskan YOLO11n. **RQ2.** Resolusi 1280 menaikkan AP@0,5 objek kecil +0,101 (+125,5%), dengan biaya latih  $2,23 \times$  dan FPS –23,9%, tetapi objek kecil tetap terlemah dan objek sedang meningkat lebih besar secara absolut.

**Keterbatasan.** (i) Latih hanya 2.000 dari 6.471 citra dan 30 epoch; epoch terbaik adalah epoch 30 pada kedua kondisi, sehingga belum konvergen dan 1280 mungkin berbeda bila dilatih lebih lama. (ii) Satu seed dan satu run per kondisi; tidak ada selang kepercayaan, dan GPU tidak sepenuhnya deterministik. (iii) Batch 2 dan hanya YOLO11n. (iv) Tidak ada ablasi yang memisahkan resolusi piksel, kerapatan grid, dan kapasitas model. (v) Ukuran objek memakai piksel citra asli yang bervariasi, dan ukuran bercampur dengan kelas. (vi) Kasus gagal dipilih dari yang paling ekstrem dan sebagian dari satu urutan video. (vii) Maksimum 500 deteksi per citra, dengan citra test terpadat memuat 461 GT. (viii) Angka mAP berbeda antar implementasi (Tabel V); angka per ukuran memakai pycocotools.

**Saran.** Pelatihan pada seluruh data dan lebih banyak epoch; beberapa seed dengan *bootstrap* atas citra; resolusi antara (800, 960) serta latih 640 dan uji 1280 untuk memisahkan efek; varian lebih besar (11s, 11m); inferensi berbasis irisan seperti SAHI [6]; dan re-implementasi YOLOv1 sebagai baseline struktural.

## REFERENSI

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [2] D. Du *et al.*, “VisDrone-DET2019: The vision meets drone object detection in image challenge results,” in *Proc. IEEE/CVF ICCV Workshops*, 2019.
- [3] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014, pp. 740–755.
- [5] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
- [6] F. C. Akyon, S. O. Altinuc, and A. Temizel, “Slicing aided hyper inference and fine-tuning for small object detection,” in *Proc. IEEE ICIP*, 2022, pp. 966–970.